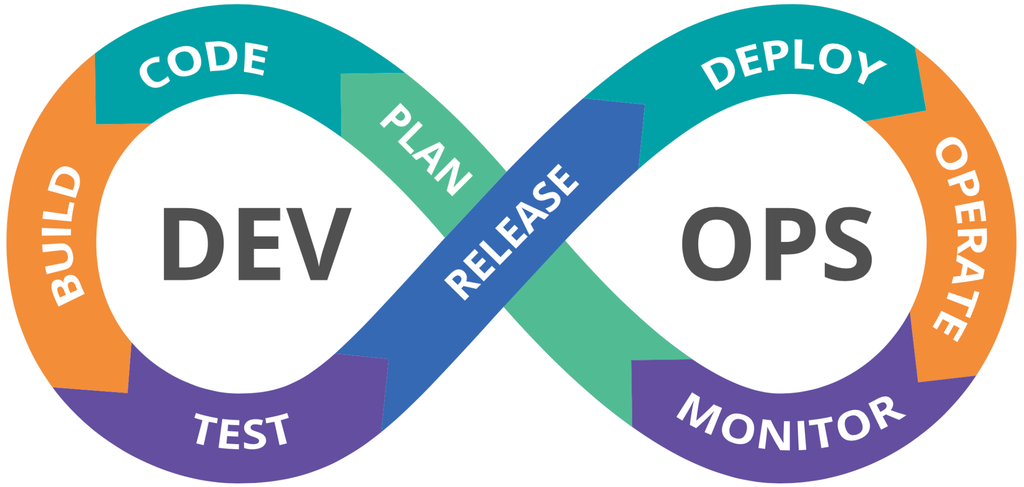
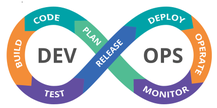



Tehničke prakse za uspostavljanje povratnih tokova
Kreiranje odgovarajuće telemetrije koja omogućava da se problemi uoče i reše

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis



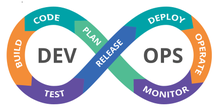

# Tehničke prakse za uspostavljanje povratnih tokova

- Ove tehničke prakse treba da pomognu da se uspostavi dobar tok povratnih informacija iz operativnih timova ka razvojnim kako bi se obezbedilo da se svi problemi uoče što pre i da se te informacije odmah učine dostupne svima kojima je to neophodno.

- Ovo omogućava da se problemi brzo lociraju i koriguju, idealno mnogo pre nego što bi takvi problemi mogli proizvesti veće ili katastrofalne otkaze.

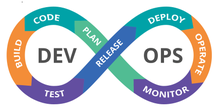
- Formira se i radno okruženje koje obezbeđuje da se znanje koje nastaje u operativnim timovima pri eksploataciji aplikacije prenosi u razvojne timove i upravljanje projektom i gde se ono koristi da se unaprede razvojni procesi.

- Svakom učesniku u lancu razvoja omogućava se da dobije povratne informacije o onome što je radio.




# Tehničke prakse za uspostavljanje povratnih tokova

- Tehničke prakse koje omogućavaju uspostavljanje kvalitetnih povratnih informacija podrazumevaju:
  - **kreiranje odgovarajuće telemetrije koja omogućava da se problemi uoče i reše**
  - korišćenje telemetrijskih podataka za bolje predviđanje ponašanja sistema i dostizanja ciljeva
  - integrisanje korisničkih iskustava i povratnih informacija u rad razvojnih timova
  - obezbeđivanje povratnih informacija koje omogućavaju razvojnim i operativnim timovima da bezbedno isporučuju i instaliraju softver
  - obezbeđivanje povratnih informacija za povećanje kvaliteta putem revizije koda od strane kolega i programiranja u parovima

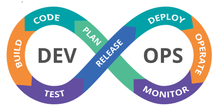



# Kreiranje odgovarajuće telemetrije

- Za operativne timove jednostavna istina je da će kompleksni sistemi u nekom momentu otkazati - male promene instalirane u sistem mogu dovesti do nepredviđenih otkaza.

- Kada se otkazi dese u produkcionim okruženjima nisu nam uvek na raspolaganju sve informacije koje bi nam pomogle da lociramo uzrok i otklonimo problem. Na primer u početnom momentu velikog otkaza, ne možemo sa sigurnošću utvrditi da li je problem u samoj našoj aplikaciji, u platformi ili u otkazu nekog eksternog sistema.

- Rudimentarni pristup operativnih timova mogao bi da bude “restartuj server, ako ne uplai restartuj onaj sledeći server, ako ni to ne radi restartuj sve servere, ako ni to ne proradi optuži programere - opet su nešto zabrljali”



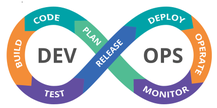

# Kreiranje odgovarajuće telemetrije

- Ali u realnosti pokazuje se da kompanije koje pružaju najviši nivo usluga (najveću dostupnost svojih servisa) restartuju svoje servere i do 20 PUTA ređe nego firme koje često imaju problema.

- Ovakve kompanije zapravo mnogo bolje detektuju, dijagnostikuju i rešavaju probleme primenjujući “kulturu uzročno posledičnih veza” (*culture of causality)* [1]

  - Kompanije koje pokazuju visoke performanse primenjuju disciplinovan pristup rešavanju problema, oslanjajući se na telemetriju iz produkcionih okruženja kako bi dobro razumeli koji su sve faktori doprineli razvoju problema i usmerili rešavanje problema na pravi uzrok. Nasuprot tome stoji neusmereni i nekontrolisani postupak nasumičnog restartovanja servera i “pogađanja” uzroka.

[1] *Kevin Behr, Gene Kim, and George Spafford “The Visible Ops Handbook”*




# Kreiranje odgovarajuće telemetrije

- Da bi se obezbedio ovakav princip rešavanja problema neophodno je dizajnirati sistem tako da on kontinualno proizvodi telemetrijske podatke. Telemetrija se može opisati kao automatizovani proces u kome se rezultati merenja i drugi podaci prikupljaju sa udaljenih tačaka i prenose na prijemno mesto gde se obrađuju i služe za nadzor sistema (monitoring)

- Jedan od ciljeva organizovanja radnih procesa treba da bude i kreiranje odgovarajućih telemetrijskih podataka, kako iz samih aplikacija koje razvijamo tako i iz okruženja u kome se one izvršavaju (predprodukcionih i produkcionih), tako i iz same linije za isporuku (*deployment pipeline*).




# Kreiranje odgovarajuće telemetrije - iskustvo kompanije Etsy

- Kompanija je prolazila kroz DevOps transformacioni period u 2009.

- Mnoštvo različitih aplikacija su transformisali u LAMP stek, zamenjujući pri tome i kritične delove svojih sistema, vodeći računa da klijenti ne osete promene.

- Brzo su shvatili da im je neophodno mnogo telemetrijskih podataka kako bi bili sigurni da se njihove aplikacije po instalaciji ponašaju onako kako se očekuje (ubrzo su koristili alate za prikupljanje informacija sa servera, vizualizaciju i agregaciju raznih metrika, i instalirali su ekrane da svi u razvojnom timu mogu da prate ponašanje aplikacija)

- Omogućivši programerima da dodaju za njih relevantu telemetriju u svoj  kod, već 2011 kompanija je pratila oko 200.000 raznih indikatora na različitim nivoima svog tehnološkog steka (aplikativne funkcije, zdravlje aplikacije, baze, OS, stanje *storage-a,* mreže…). Trideset najbitnijih indikatora je bilo uvek vidljivo na *“deployment dashboardu”.*

# Uticaj telemetrijskih podataka na brzinu rešavanja problema

- U izveštaju *State of DevOps 2015* godine pokazano je da kompanije visokih performansi (one koje se oslanjaju na telemtrijske podatke i analizu uzroka) rešavaju probleme i do 168 puta brže od onih sa niskim performansam.

- Vreme oporavka se često meri samo minutama, dok se kod kompanija sa niskim performansama vreme oporavka često meri i danima.

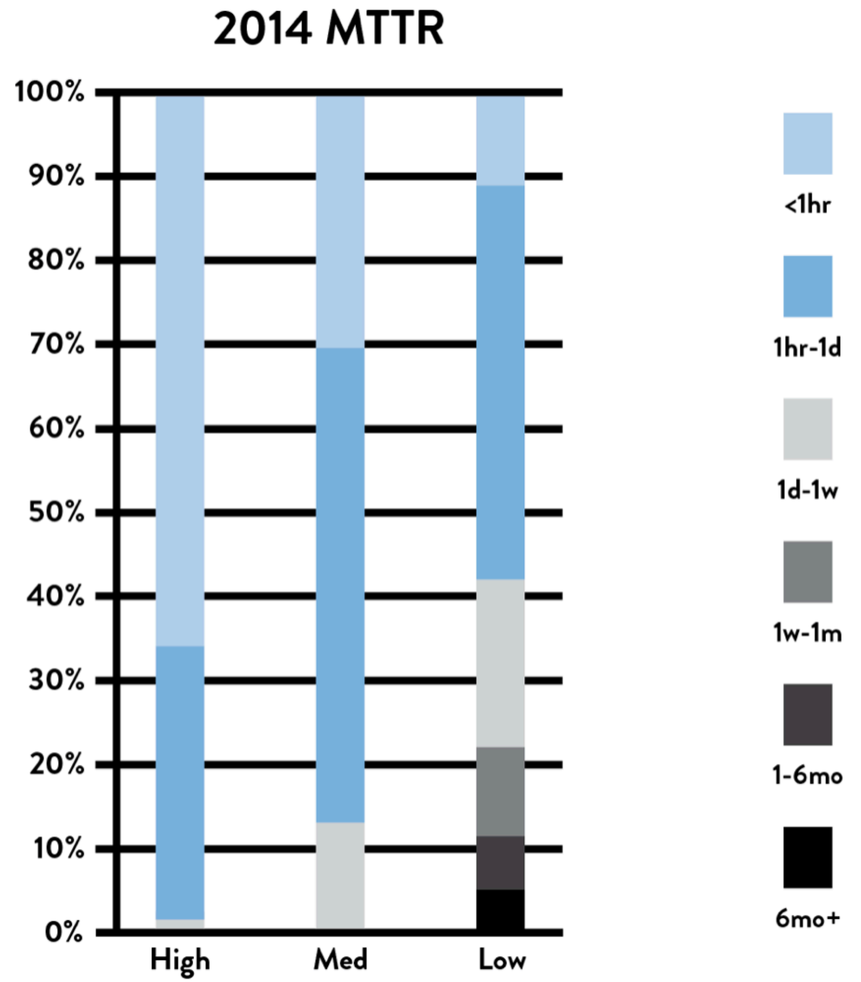
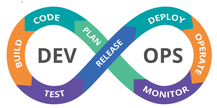



**Figure 25:** *Incident resolution time for high, medium, and low performers*



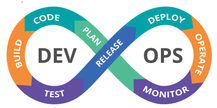

# Uspostavljanje centralizovane infrastrukture za telemetriju

- Operativni nadzor softverskih sistema nije nova stvar. Postoje razni alati koji omogućavaju nadzor instaliranih sistema (HP OpenView, IBM Tivoli, MS SCOM…)

- Prikupljanje informacija sa nadziranih sistema može biti realizovano preko agenata koji se instaliraju na nadzirane sisteme i koji šalju podatke centralnom sistemu, ili preko neagentskih tehnologija.

- Osim toga praksa logovanja kritičnih operacija i posebno grešaka u aplikacijama omogućava da se logovi koriste kao izvor kvalitetnih informacija o stanju i ponašanju sistema.

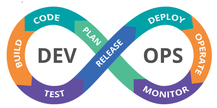
- Međutim često se dešavalo da programeri loguju informacije koje su njima zanimljive, a operativni timovi nadziru na primer samo platformu i da li su serveri aktivni ili ne. Praktično se formiraju izolovani skupovi informacija i u slučaju nekog otkaza nijedna ni druga strana u takvoj situaciji možda neće biti u stanju da izoluje uzrok.




# Uspostavljanje centralizovane infrastrukture za telemetriju

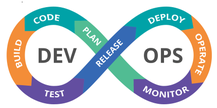
- Aplikacije se moraju dizajnirati i razvijati tako da generišu dovoljno relevantnih telemetrijskih podataka, na svim nivoima aplikacije i infrastrukture, kako bi se omogućilo praćenje i vizuelizacija informacija.

- Velike kompanije (Google, Amazon, Facebook…) u svojoj arhitekturi često koriste i open source alate (Nagios, Zenoss) koje je bilo lakše prilagoditi specifičnim uslovima nego komercijalne alate.



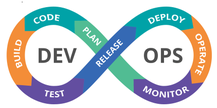

# Uspostavljanje centralizovane infrastrukture za telemetriju

- Arhitektura za prikupljanje i obradu telemetrijskih podataka treba da sadrži:
  - **Prikupljanje podataka iz sloja poslovne logike, aplikacija i sloja izvršnog okruženja** (u kome se izvršavaju naše aplikacije).
    - Iz svakog od ovih slojeva telemetrijski podaci se prikupljaju u formi događaja, loogova i metrika (indikatora).
    - Logovi mogu da se skladište i na putanjama specifičnim za određene aplikacije ili sisteme, ali je poželjno da se oni šalju u neki zajednički servis koji omogućava centralnu obradu, rotaciju logova i brisanje. Ove funkcionalnosti često obezbeđuju i sami OS (syslog, Event Log…)
    - Na nivou aplikacija se mogu prikupljati metrike koje pomažu da bolje razumemo kako se same aplikacije ponašaju.
    - Na nivou operativnog sistema moguće je prikupiti metriku o korišćenju procesora, memorije, diskova korišćenjem raznih alata (collectd, Ganglia). Za prikupljanje podataka o indikatorima performansi na raspolaganju su i drugi alati poput AppDynamics, New Relic…




# Uspostavljanje centralizovane infrastrukture za telemetriju

- **Ruter za događaje** koji je odgovoran za prikupljanje informacija o događajima i čuvanje metrike.
  - potencijalno obezbeđuje i alate za vizuelizaciju, praćenje trendova, alarmiranje, detekciju anomalija
  - prikupljanje, čuvanje i agregiranje telemetrijskih podataka omogućava i naknadne analize i proveru “zdravlja” komponenti
  - obično se na ovim komponentama podešava i konfiguracija sistema (npr. prag za alarme po određenim indikatorima)
  - centralizovani logovi mogu se transformisati u metrike (npr. brojanjem pojava logova određenog tipa), a ovako dobijene metrike se posle mogu koristiti i za statističku obradu i na primer otkrivanje anomalija
  - osim telemetrije iz našeg sistema neophodno je prikupljati podatke i iz *deployment pipeline-a* (npr. broj testova koji padaju, trajanje builda ili testova i sl.)



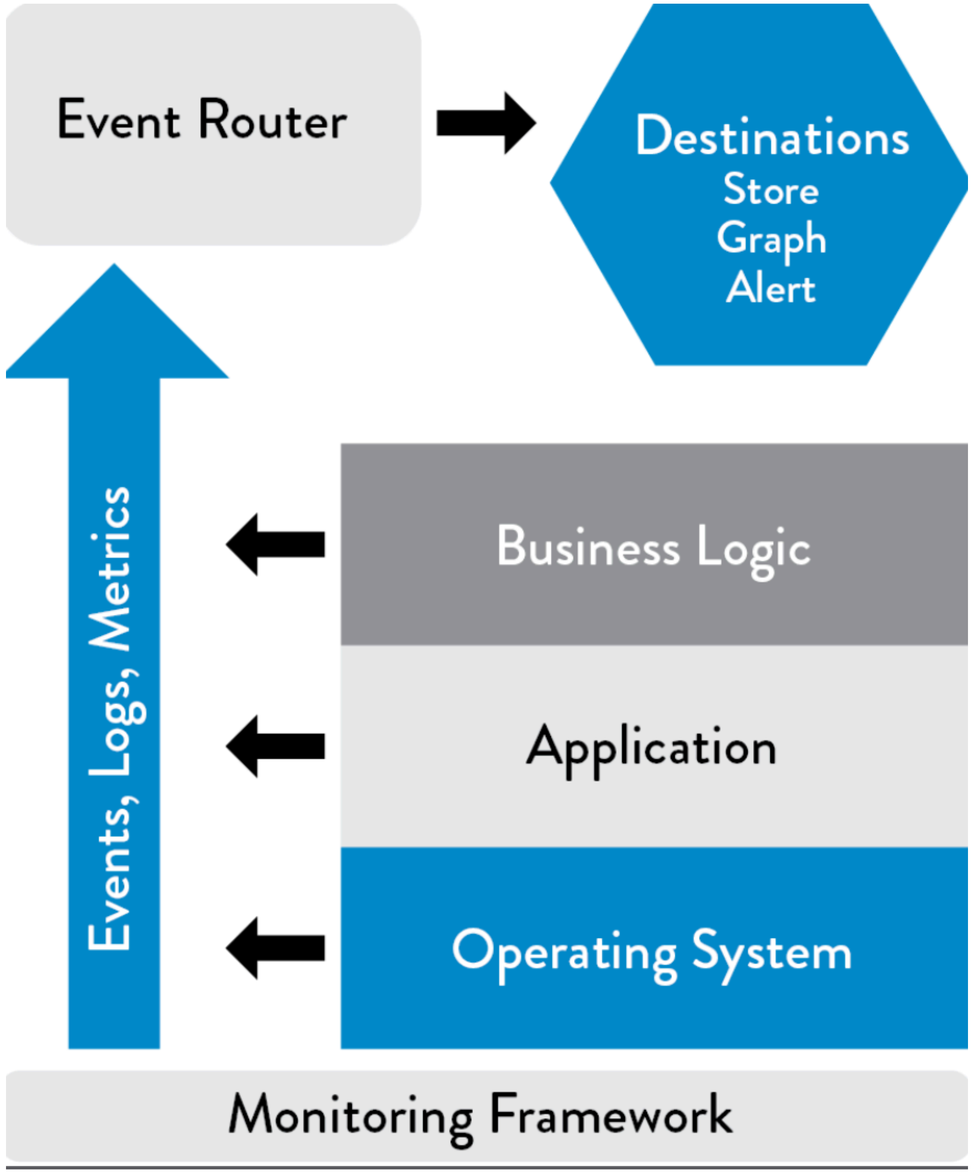

# Uspostavljanje centralizovane infrastrukture za telemetriju

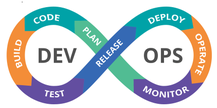
- Pristup prikupljenim podacima mora biti jednostavan, poželjno putem odgovarajućeg API-ja

- U idealno postavljenim konfiguracijama, telemetrijski podaci bi nam trebali odmah ukazati kada se dešava nešto na šta treba obratiti pažnju.

- Važnost nadzora instaliranih sistema je tolika da je jako bitno da su monitoring alati dostupni i skalabilni barem koliko i sami sistemi koje nadziru (pad monitoringa nas ostavlja u stanju da smo potpuno nesvesni šta se dešava sa našom aplikacijom).

# Uspostavljanje centralizovane infrastrukture za telemetriju



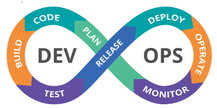



# Uspostavljanje aplikativnih logova koji pomažu u produkciji

- Kada postoji uspostavljena infrastruktura za prikupljanje telemtrijskih podataka, moramo se pobrinuti da naša aplikacija proizvodi relevantne podatke.

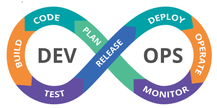
- Nažalost retko se pri razvoju softvera odmah razmišlja o ovome.

- U razvijanim aplikacijama svaka funkcionalnost bi trebalo da je “instrumentalizovana” (da je praćena odgovarajućim logovanjem podataka) - ako je bila dovoljno bitna da se uloži vreme u njen razvoj, onda je dovoljno bitna i da se nadzire njen rad.



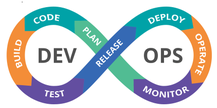

# Uspostavljanje aplikativnih logova koji pomažu u produkciji

- Prikupljene telemetrijske podatke iz aplikacije različiti timovi mogu koristiti na različite načine - programer može privremeno generisati veću količinu podataka da bi lakše locirao problem u kodu, član operativnog tima koristiće ih da detektuje problem u produkciji, dok npr. nadzorni timovi mogu da koriste ove podatke da se uvere da sistem radi u skladu sa zadatim parametrima.




# Uspostavljanje aplikativnih logova koji pomažu u produkciji

- Kako bi se obezbedila mogućnost pregleda informacija na različitim nivoima logovi su organizovani tako da sadrže događaje različitog nivoa:
  - DEBUG - informacije na ovom nivou često su vrlo detaljne i korisne za lociranje grešaka, i obično se događaji ovog nivoa isključuju na produkciji
  - INFO - informacije o događajima koje uzrokuju korisnici ili su sistemski specifični (npr. započeta transakcija…)
  - WARN - informacije o događajima koji potencijalno mogu prerasti u  grešku (npr. prespor odziv baze koji bi mogao uskoro rezultavati timeoutom). Događaji ovog nivo često “podižu” alarm (još uvek nam omogućavaju preventivno delovanje)
  - ERROR - informacije o greškama koje su se desile otkom izvršavanja programa (API greške, interne greške…)
  - FATAL - informacije o događajima koji uslovljavaju terminiranje programa (npr. server pokušao da zauzme već zauzeti port)




# Uspostavljanje aplikativnih logova koji pomažu u produkciji

- Svi potencijalno bitni događaji u aplikaciji bi trebali generisati log zapise. Anton A. Chuvakin (research VP at Gartner's GTP Security and Risk Management group) predložio je listu događaja koje obavezno treba logovati:
  - Authentication/authorization decisions (including logoff)
  - System and data access
  - System and application changes (especially privileged changes)
  - Data changes, such as adding, editing, or deleting data
  - Invalid input (possible malicious injection, threats, etc.)
  - Resources (RAM, disk, CPU, bandwidth, or any other resource that has hard or soft limits)
  - Health and availability
  - Startups and shutdowns
  - Faults and errors
  - Circuit breaker trips
  - Delays
  - Backup success/failure




# Korišćenje prikupljene telemetrije za rešavanje problema

- Disciplinovano rešavanje problema poboljšava odziv u slučaju problema.

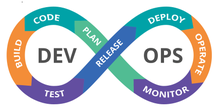
- Haotičan pristup i nagađanje šta bi mogao biti uzrok mnogo češće vodi do uspostavljanju negativne metrike - koliko vremena je potrebno za “nisam ja kriv” (*mean time until declared innocent)*

- Dobro uspostavljen sistem prikupljanja telemetrije omogućava sistematičan pristup rešavanju problema:
  - Koje dokaze imamo na osnovu podataka da se problem (upravo) dešava?
  - Koji događaji ili promene u našoj aplikaciji ili okruženju mogu doprinostiti ispoljavanju problema?
  - Koju hipotezu možemo postaviti da ona objašnjava vezu između pretpostavljenog uzroka i uočenih problema?
  - Kako možemo na osnovu podataka dokazati neku od ovih hipoteza i rešiti problem?




# Korišćenje prikupljene telemetrije za rešavanje problema

- Kada na raspolaganju imamo dovoljno telemetrijskih podataka i rešavanje problema zasnovano na merljivim podacima, ne samo da imamo bolji MTTR, već se uspostavlja i bolja saradnja između operativnih i razvojnih timova.




# Kreiranje produkcionih telemetrijskih podataka

- Da bi se omogućilo brzo i efikasno pronalaženje i rešavanje problema neophodno je da svi učesnici imaju mogućnost da kreiraju relevantne telemetrijske podatke tokom svog svakodnevnog posla i da ih mogu lako prikupiti, vizuelizovati i analizirati.

- Ovo podrazumeva odgovarajuću infrastrukturu i bilioteke koje će omogućiti lako dodavanje telemetrije za bilo koju funkcionalnost koja je nekome od interesa.

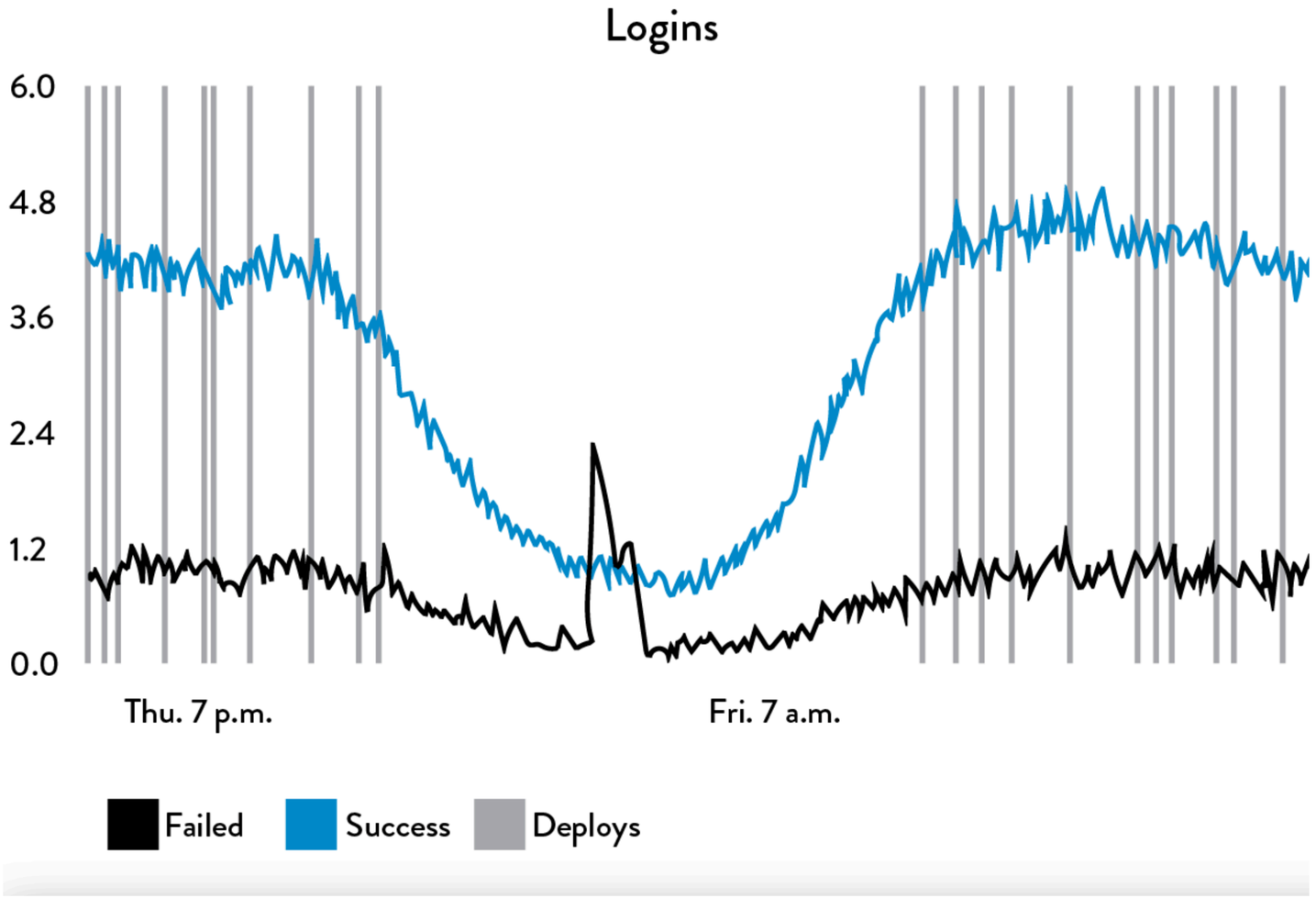
- StatsD je jedna od često korišćenih biblioteka koja omogućava da se sa samo jednom linijom dodatog koda generišu timeri i brojači (dostupno za Ruby, Java, Perl, PHP, Python…). Vrlo često se koristi u kombinaciji sa Graphite i Grafana bibliotekama.

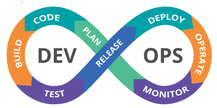
- Alternative za StatsD JMX, codahale metrics, a i drugi alati poput New Relic, AppDynamics, and Dynatrace omogućavaju prikupljanje metrike iz aplikacija.

# Kreiranje produkcionih telemetrijskih podataka

- Primer:
- jednostavno dodvanja brojača za evidentiranje uspešnosti logovanja.

StatsD::increment(“login.successes”))





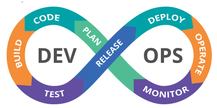

# Obezbeđivanje pristupa telemetrijskim podacima po potrebi i širenje informacija

- ključna ideja je da se učesnicima u lancu razvoja omogući pristup prikupljenim telemetrijskim podacima o stanju ili performansama određenih servisa

- ovaj pristup treba da je takav da ne zahteva administratorska ili privilegovana prava niti manuelne dozvole za pristup (otvaranje ticketa na servis centru)

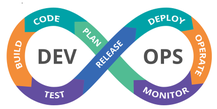
- želimo da su telemetrijski podaci lako i jasno vidljivi svim članovima tima

- *information radiator*, definisan od strane Agile Alliance kao “generički termin za bilo koji vid ručno pisanih, crtanih, štampanih materijala ili prikaza u elektronskom formatu koji timovi imaju na vidnom mestu tako da svi članovi tima ili posetioci imaju relevantne informacije dostupne na prvi pogled”




# Obezbeđivanje pristupa telemetrijskim podacima po potrebi i širenje informacija

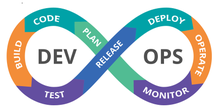
- korišćenje *information radiator-a* promoviše i odgovornost u timu:
  - tim nema šta da krije od posetioca (klijenata, zainteresovanih…)
  - tim ne skriva informacije ni od samog sebe - problemi se jasno uočavaju i rešavaju
- Postoji i dosta otpora ka potpunoj transparentnosti (da li se zaista svi problemi odmah javljaju krajnjem korisniku?)




# Identifikovanje nedostajućih telemetrijskih podataka i njihova dopuna

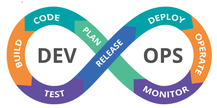
- Neophodno je analizirati i identifikovati koji podaci nedostaju u telemetriji da bi se neke funckionalnosti ili performanse uspešno pratile

- Nedostatak informacija može da umanji našu sposobnost uočavanja, lociranja i otklanjanja grešaka




# Identifikovanje nedostajućih telemetrijskih podataka i njihova dopuna

- Podaci nam trebaju sa različitih (već spominjanih nivoa):
  - nivo poslovne logike (na primer ukupan broj transakcija, ukupan ostvareni promet, rezultati A/B testiranja…)
  - aplikativnog nivoa (na primer vreme obrade transakcije, aplikativne greške…)
  - infrastrukturnog nivoa (količina saobraćaja ka serveru, opterećenje procesora…)
  - klijentskog sloja (npr JavaScript - statistike izvšavanja određenih poziva…)
  - nivoa *deployment pipeline-a (*statusi testova, trajanje deploymenta, frekvencija deploymenta…)
- Prikupljanje podataka sa svih ovih nivoa omogućava nam detaljan uvid u “zdravlje” i performanse sistema kao celine

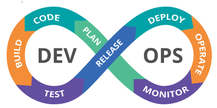



# Aplikativna i poslovna metrika

- Na aplikativnom nivou metrika koju prikupljamo ne treba da se koncentriše samo na “zdravlje” aplikacije (npr. korišćenje memorije, vreme izvršavanja i broj transakcija)

- Jako su bitni i podaci koji pokazuju koliko dobro aplikacija postiže poslovni cilj (npr. broj novoregistrovanih korisnika, broj naručenih proizvoda, koliko često se koriste određene funkcije sistema…)

- Na primer, na komercijalnom web sajtu jako je bitno prikupiti informacije o akcijama koje je korisnik obavio i da li su one dovele do uspešnih transakcija (kupovine), jer se time može uočiti da li neki sadržaji privlače ili odbijaju kupce. One nam govore kako da prilagodimo aplikaciju da povećamo poslovni učinak.



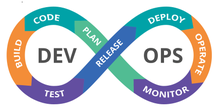

# Infrastrukturna metrika

- Cilj je da iz svakog okruženja na kome se pokreće naša aplikacija prikupljamo dovoljno indikatora koji nam mogu pomoći da unapred predvidimo potencijalne probleme i da utvrdimo da li sama infrastruktura utiče na pojavu problema (nedovljan kapacitet memorije, ili preveliko opterećenje procesora npr.)

- Dobra metrika treba da nam omogući da utvrdimo koji deo infrastrukture je kritičan (baza, OS, kapacitet diskova, premala propusna moć mreže…)

- Dosta dugo je uspostavljanje veze između infrastrukture i servisa bilo prepušteno manuelnom konfigurisanju, sada imamo alate koji omogućavaju da se servisi automatski registruju i pronalaze (Zookeeper, Consul).

- Automatska registracija servisa i relevanti podaci (broj aktivnih instanci, uspešna registracija nove instance ird.), takođe mogu da se koriste kao infrastrukturna metrika




# Dodavanje pomoćnih informacija na metriku

- Ponekad je potrebno učiniti vidljivim i neke informacije koje ne nastaju u samom sistemu, ali bitno utiču na dostupnost određenih usluga ili delova sistema, kao što su na primer aktivnosti na održavanju koje privremeno menjaju stanje nekih servisa iz aktivan u “redovno održavanje”.